สัปดาห์ที่ 2 · 03_MultiAccount_Container

การสร้าง Docker image สำหรับระบบหลายบัญชี จาก Dockerfile

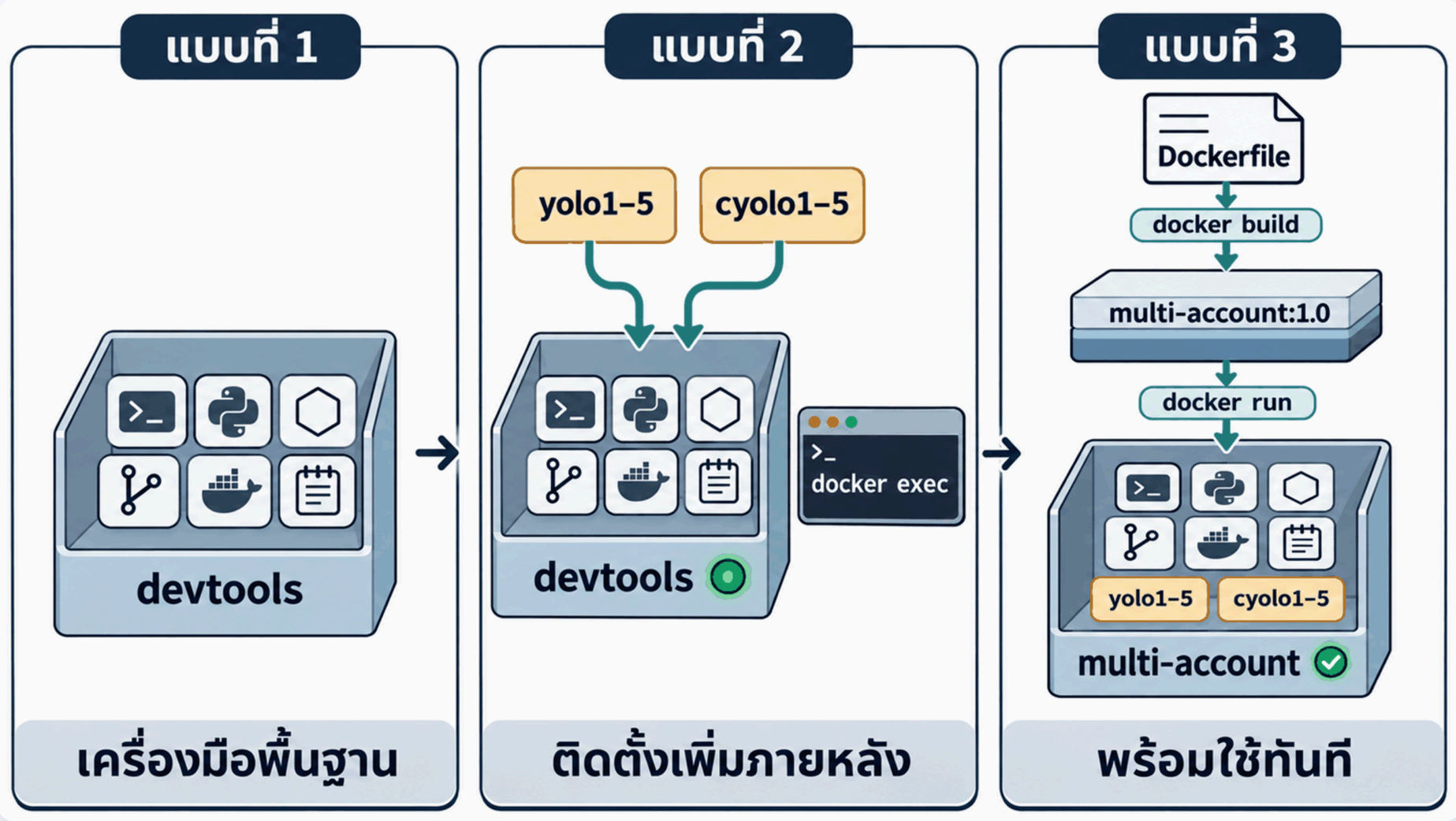
สั่งงานทุกขั้นจาก Claude Code app

multi-account:1.0 · yolo1–5 · cyolo1–5 · Playwright MCP · volume ai-accounts



1 · เปรียบเทียบ

การเตรียมสภาพแวดล้อม 3 แบบใน 01_week02



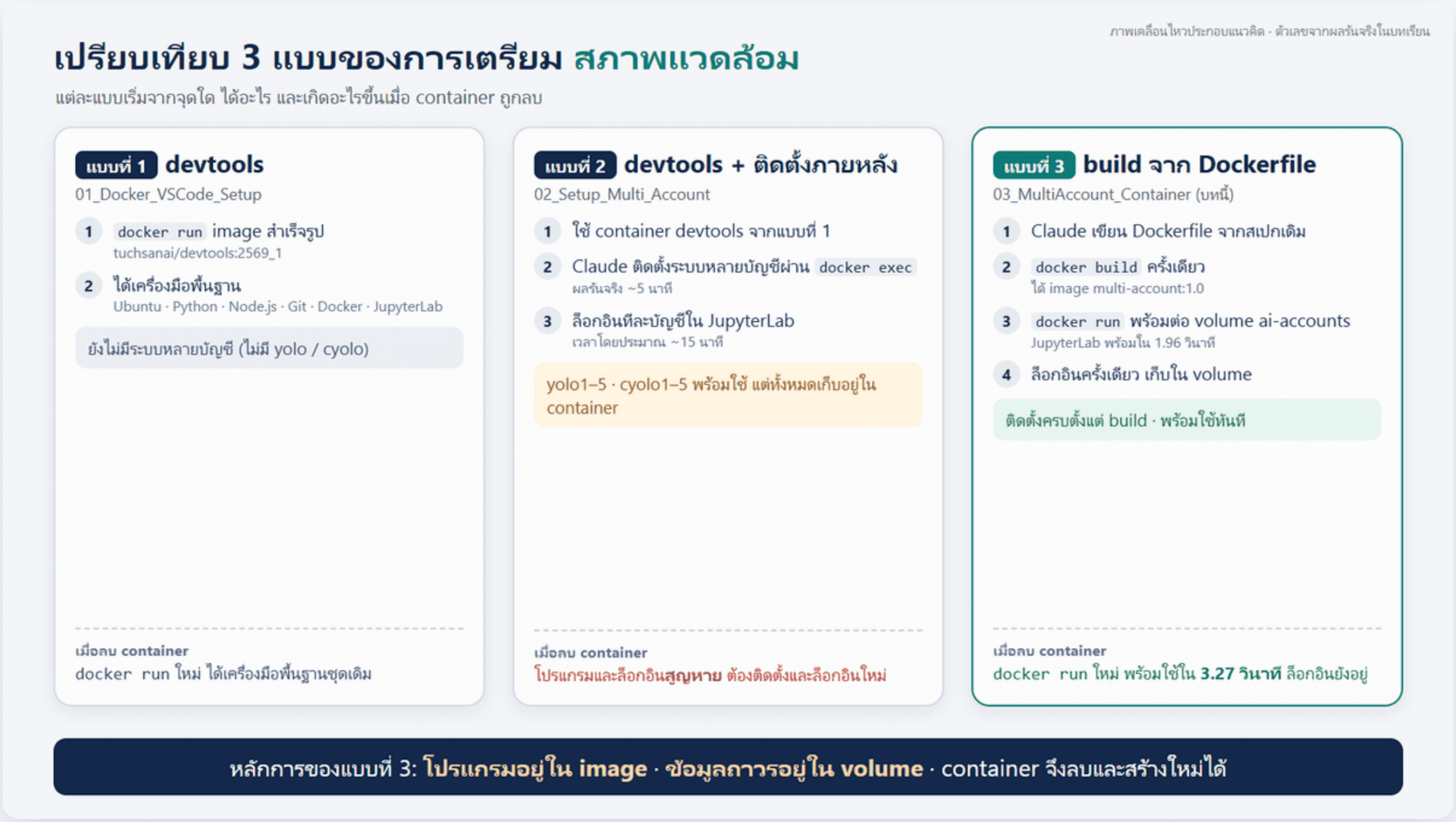
1 · เปรียบเทียบ

แต่ละแบบต่อยอดจากแบบก่อนหน้า

	แบบที่ 1 · devtools	แบบที่ 2 · ติดตั้งภายหลัง	แบบที่ 3 · build จาก Dockerfile
บทเรียน	01_Docker_VSCode_Setup	02_Setup_Multi_Account	**บทนี้**
จุดเริ่มต้น	image `tuchsanai/devtools:2569_1`	container `devtools`	Dockerfile
ระบบหลายบัญชี	ไม่มี	ติดตั้งผ่าน `docker exec`	ติดตั้งตั้งแต่ขั้น build
ล็อกอินเก็บที่	–	ภายใน container	volume `ai-accounts`
เมื่อลบ container	ได้เครื่องมือพื้นฐานชุดเดิม	ติดตั้งและล็อกอินใหม่	`docker run` ใหม่ พร้อมใช้ทันที

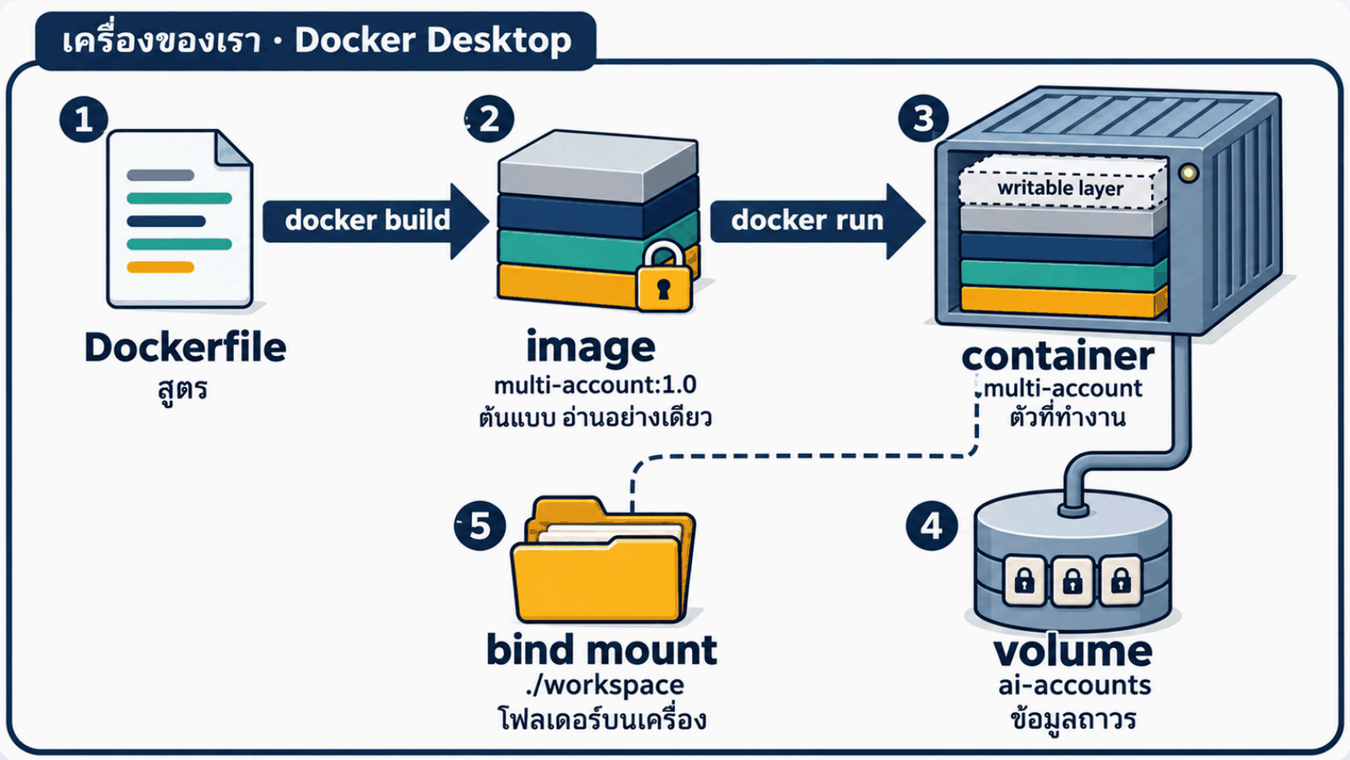
1 · เปรียบเทียบ

ผลเมื่อ container ถูกลบ



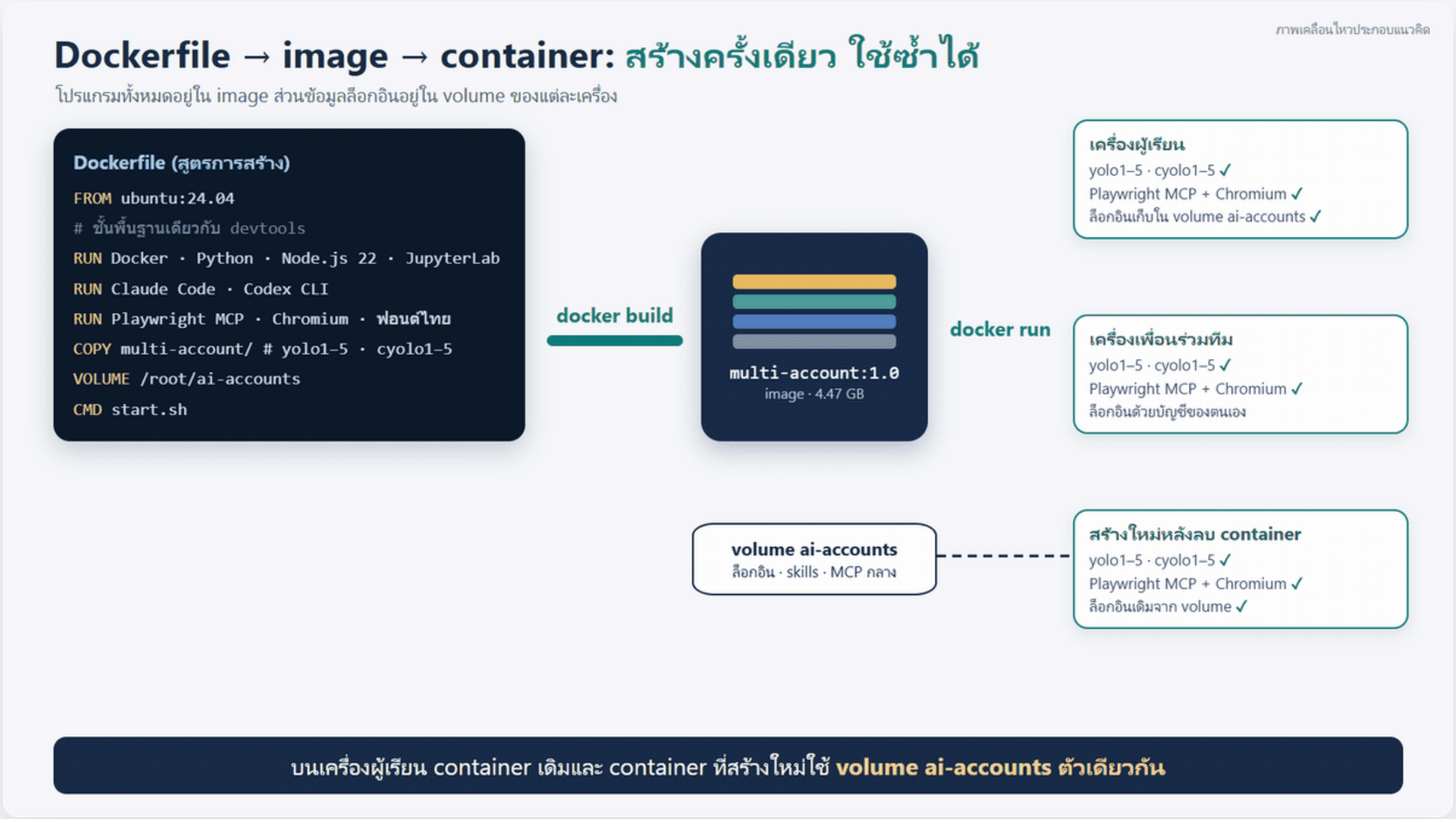
ภาพเคลื่อนไหวประกอบแนวคิด เวลาของแบบที่ 2 มาจากบท 02_Setup_Multi_Account ตัวเลขของแบบที่ 3 มาจากการรันจริงในบทนี้

2 · องค์ประกอบของ Docker



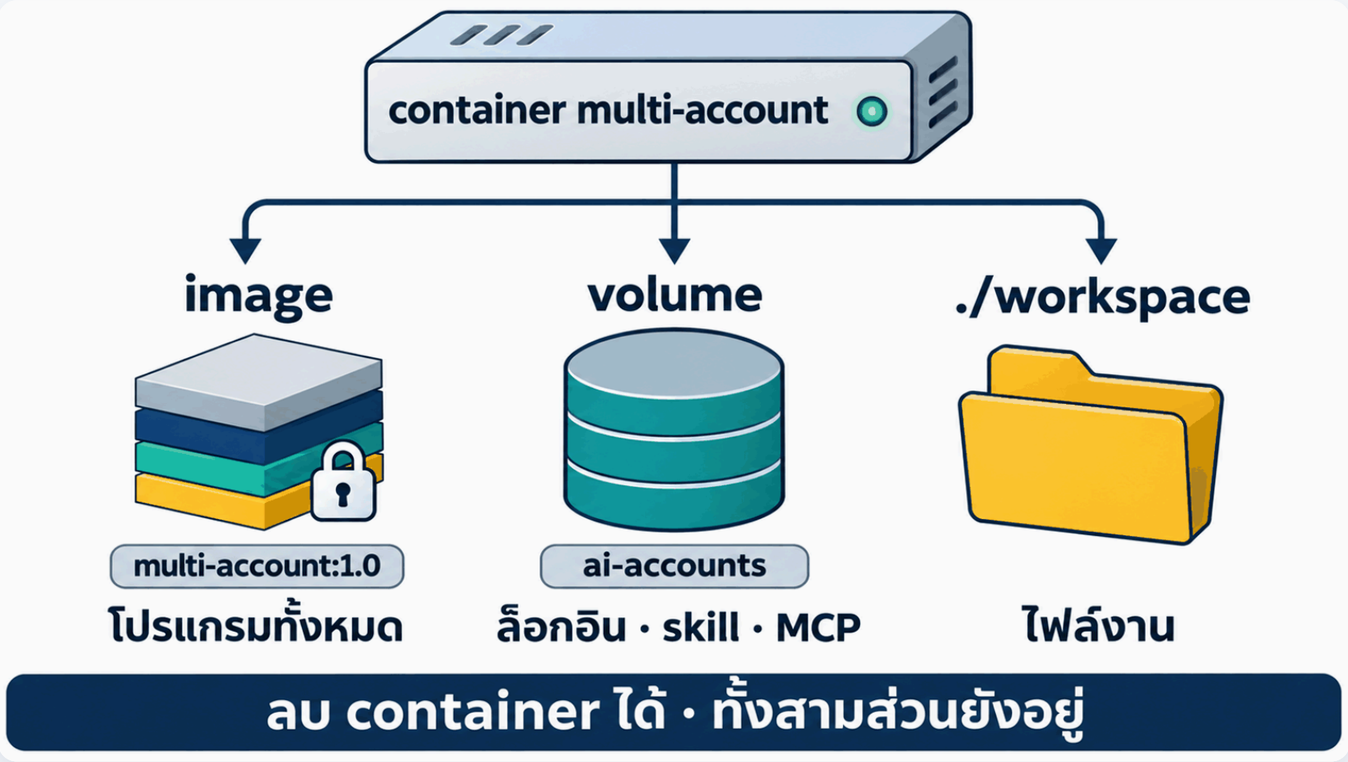
องค์ประกอบ	ในบทนี้
Dockerfile ขั้นตอนการสร้าง image	`MultiAccountDevtool/Dockerfile`
image ต้นแบบ อ่านอย่างเดียว	`multi-account:1.0`
container ตัวที่ทำงานจริง	`multi-account`
volume ข้อมูลคงอยู่แม้ลบ container	`ai-accounts`
bind mount โฟลเดอร์บนเครื่อง	`./workspace`

build ครั้งเดียว · run ได้ทุกเครื่อง



โปรแกรมชุดเดียวกันทุก container ส่วนข้อมูลล็อกอินเป็นของแต่ละเครื่อง เก็บใน volume

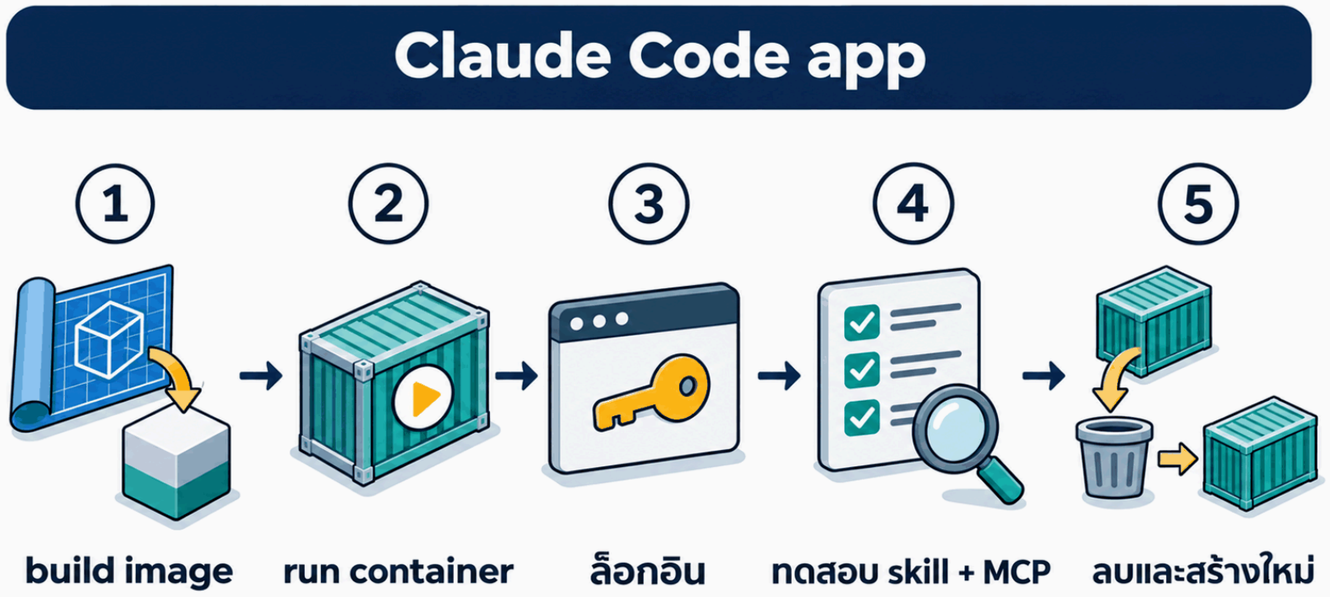
2.1 · โครงสร้างของ image



ส่วน	เมื่อลบ container
ชั้นของ image (4 ชั้น) โปรแกรมทั้งหมด	ยังอยู่
container layer ไฟล์ระหว่างทำงาน	หายไป
volume ai-accounts ล็อกอิน · skill · MCP	ยังอยู่
./workspace ไฟล์งาน · หลักฐาน	ยังอยู่

> image **ไม่มีข้อมูลล็อกอินอยู่ภายใน** จึงแชร์ให้ผู้อื่นได้โดยไม่เปิดเผยบัญชี

3 · ภาพรวมขั้นตอนการทดลอง



ขั้น	ผู้ดำเนินการ	เวลา
1 build image	Claude	~28 นาที
2 สร้าง container	Claude	~1 นาที
3 ล็อกอิน + ตรวจ	ผู้เรียน + Claude	~15 นาที
4 ทดสอบแบบขนาน	Claude	~5 นาที
5 ลบแล้วสร้างใหม่	Claude	~3 นาที

3 · ก่อนเริ่ม

ตรวจความพร้อมและเปิดโฟลเดอร์งาน

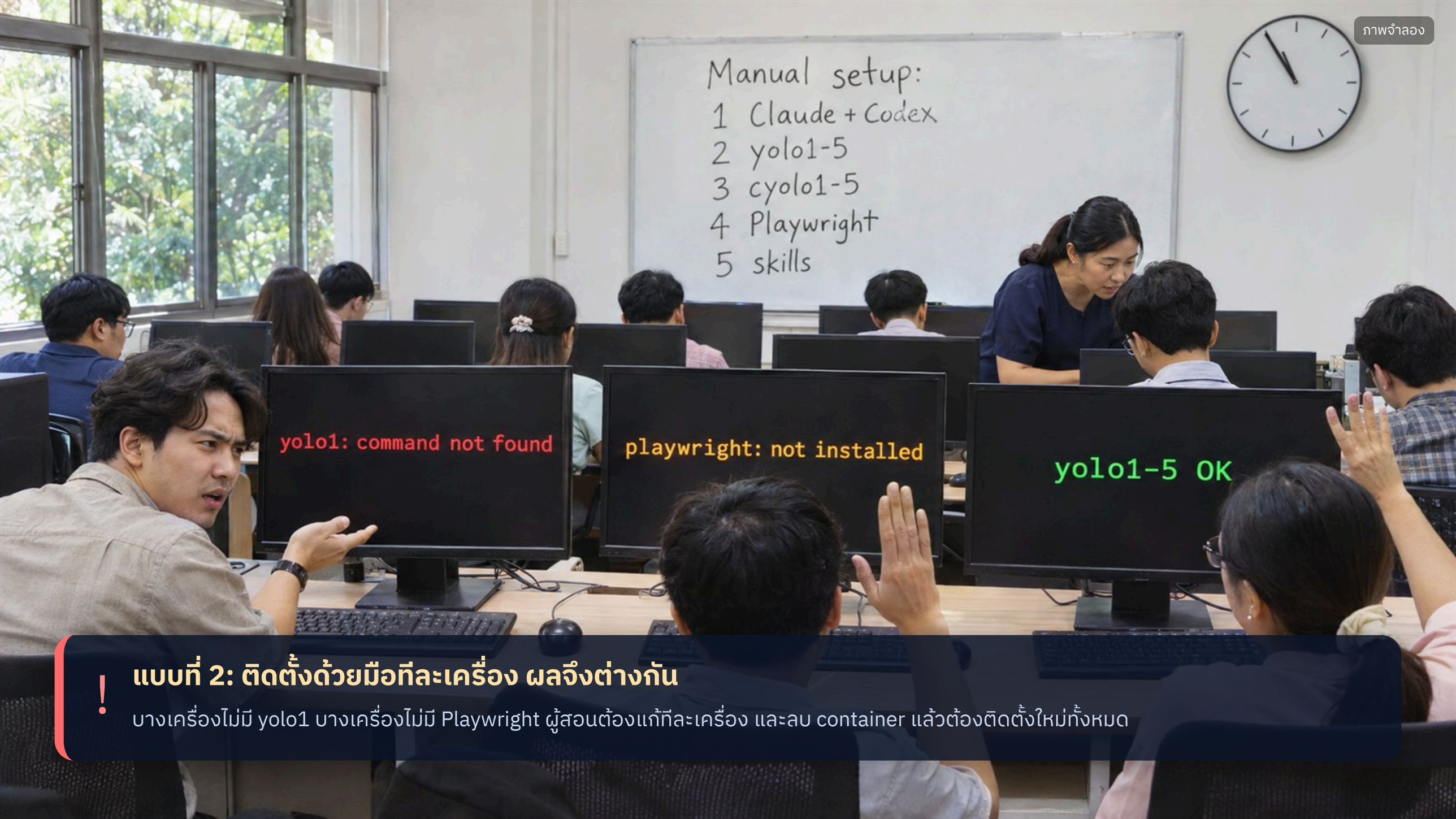
1. Docker Desktop แสดง **Engine running**
2. Claude Code app ล็อกอินแล้ว
3. พื้นที่ดิสก์ว่างอย่างน้อย ~10 GB
   image ~4.5 GB + build cache
4. **Code** → **New session** → เลือกโฟลเดอร์
   `03_MultiAccount_Container`

```
03_MultiAccount_Container/
├── 00_Devtool_Container/     ← Dockerfile มาตรฐานของ devtools
(input ที่ 1)
├── prompt_multi_account.md   ← สเปกระบบหลายบัญชี (input ที่ 2)
└── _solution/                ← เฉลยจากการรันจริง (หากต้องการให้
Claude ทำเองทั้งหมด ให้ย้ายโฟลเดอร์นี้ออกก่อน)
```

1

ให้ Claude สร้าง image

อ่านสเปก + Dockerfile มาตรฐาน → เขียน **MultiAccountDevtool/** → **docker build** → ทดสอบ (~30 นาที)

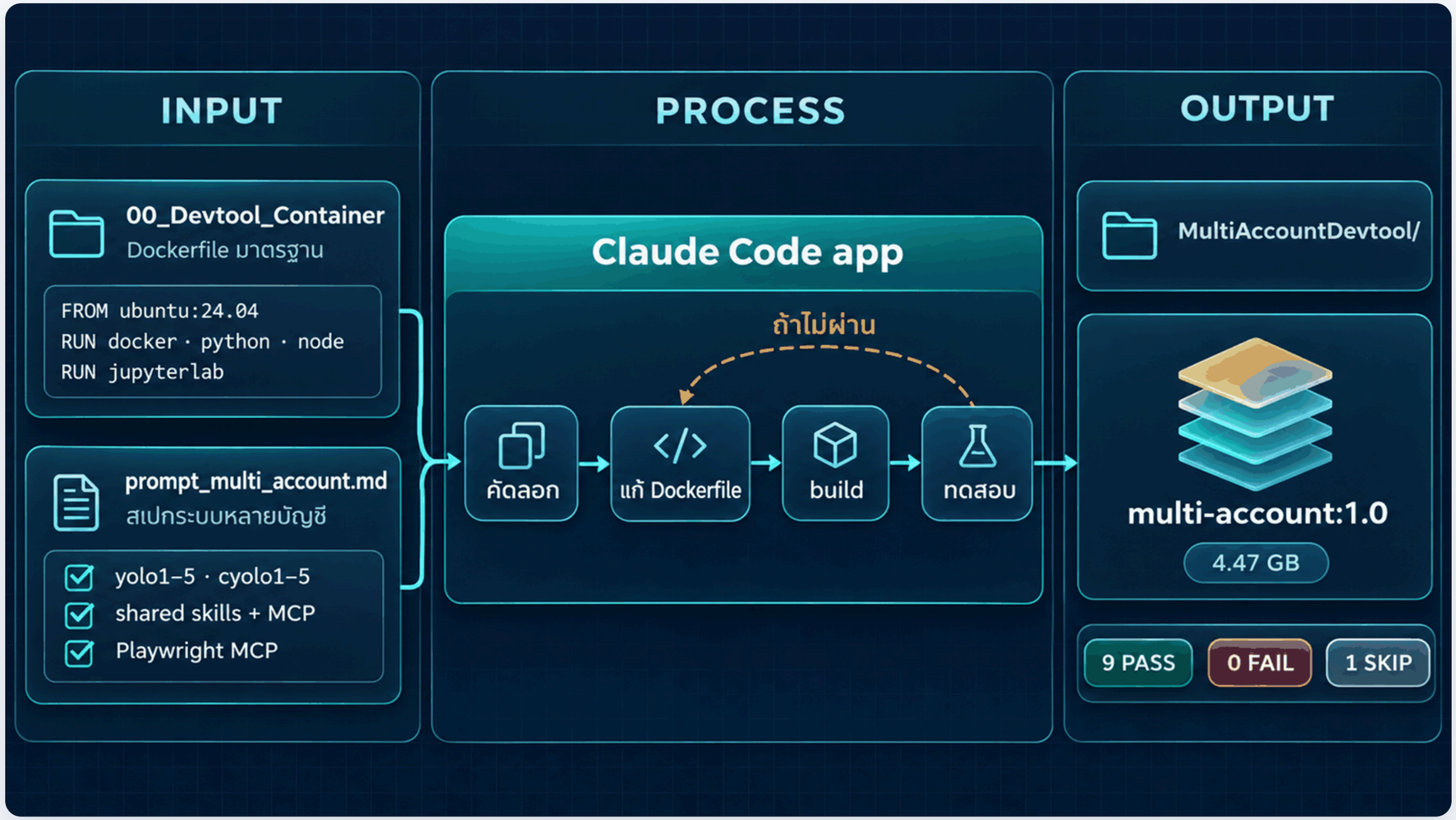


แบบที่ 2: ติดตั้งด้วยมือทีละเครื่อง ผลจึงต่างกัน

บางเครื่องไม่มี yolo1 บางเครื่องไม่มี Playwright ผู้สอนต้องแก้ทีละเครื่อง และลบ container แล้วต้องติดตั้งใหม่ทั้งหมด

ขั้นที่ 1.1 · input และกระบวนการ

เขียนรายการติดตั้งเป็นสเปก แล้ว build ครั้งเดียว



ถ้าการทดสอบไม่ผ่าน Claude กลับไปแก้ Dockerfile แล้ว build ใหม่ · ตัวเลขทั้งหมดมาจากการรันจริง

ขั้นที่ 1.2 · prompt_multi_account.md

สเปกระบบหลายบัญชีมี 5 หัวข้อ

หัวข้อ	สาระ
เป้าหมาย	`yolo1-5` (Claude Code) และ `cyolo1-5` (Codex CLI) แยกโฟลเดอร์ต่อบัญชี + `shared` สำหรับ skill และ MCP กลาง
กติกา	แยก credential ต่อบัญชี · installer รันซ้ำได้ไม่ทับของเดิม · MCP ตัวแรก Playwright · skill ตัวอย่าง `shared-check`
ข้อเท็จจริงจากการทดลอง	ข้อควรรู้เฉพาะของแต่ละ CLI ที่พบจากการรันจริง
ทดสอบ	เกณฑ์ PASS / FAIL / SKIP
รายงาน	`setup-report.md` · `login-guide.md` · `checklist.md`

นำมาจากพรอมต์ขั้นที่ 1 ของบท 02_Setup_Multi_Account บทนั้นติดตั้งลงใน container ที่กำลังทำงาน บทนี้ใช้เป็นข้อกำหนดของ image

ขั้นที่ 1.2 · ข้อความเต็มของ prompt_multi_account.md

```
ตรวจ --help ของเวอร์ชันจริงก่อนใช้ env/flag ห้ามเดา

## เป้าหมาย (BASE=/root/ai-accounts)
- yolo1..yolo5   → claude --dangerously-skip-permissions  (CLAUDE_CONFIG_DIR=$BASE/.Claude/yoloN)
- cyolo1..cyolo5 → codex --dangerously-bypass-approvals-and-sandbox  (CODEX_HOME=$BASE/.Codex/cyoloN)
- wrapper วางใน /usr/local/bin ให้พิมพ์ได้ทันที ส่งต่อ "$@" ครบ แจ้งชัดถ้าโฟลเดอร์บัญชีไม่มี ไม่พิมพ์ secret
- $BASE/.Claude/shared และ $BASE/.Codex/shared มี skills/ และ mcp/ แก้ครั้งเดียว ทุกบัญชีของ CLI นั้นเห็นตอนเปิดครั้งถัดไป

## กติกา
- แยก credential/config/session ต่อบัญชี เปิดพร้อมกันได้ ใช้ CLAUDE_CONFIG_DIR/CODEX_HOME เท่านั้น ห้ามเปลี่ยน $HOME
- Codex ใช้ file-based credential ต่อ CODEX_HOME ห้ามแชร์ credential ใน shared
- สร้าง installer เป็นไฟล์ใน container รันซ้ำได้ ไม่เขียนทับ credential/config เดิม ห้ามแก้ .bashrc/Git/SSH
- MCP ส่วนกลางตัวแรก: Playwright (@playwright/mcp) + Chromium ติดตั้งครั้งเดียว
- skill ตัวอย่าง shared-check: ตอบ SHARED_SKILL_OK-<tool>-<ค่าสุ่มตอนติดตั้ง> ซึ่งมีในไฟล์ skill เท่านั้น

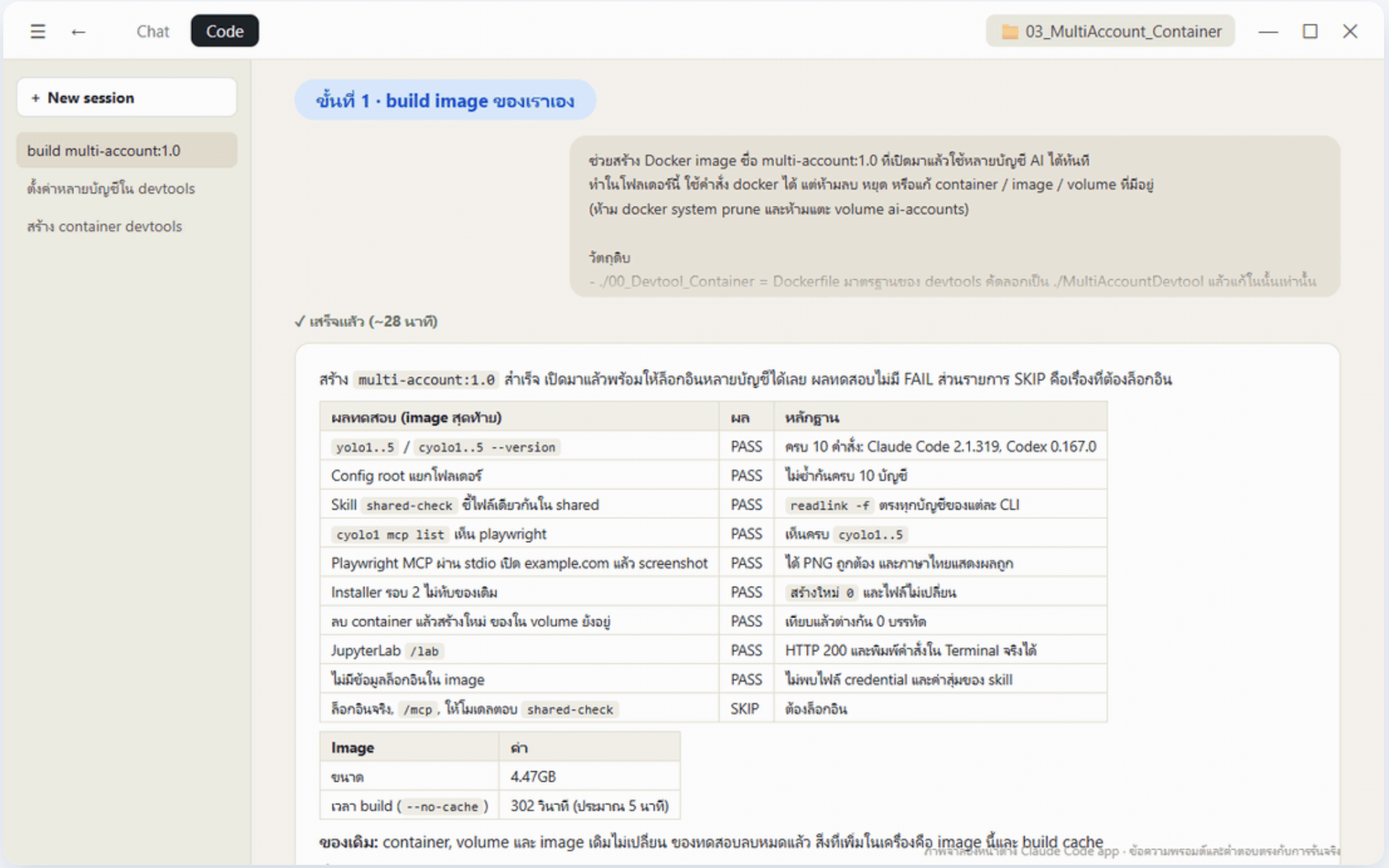
## ข้อเท็จจริงจากการทดลอง (ตรวจว่ายังจริงกับเวอร์ชันนี้)
- root ต้องตั้ง IS_SANDBOX=1; CODEX_HOME ต้อง mkdir -p ก่อน
- Claude: skill อ่านจาก $CLAUDE_CONFIG_DIR/skills; MCP ส่งผ่าน --mcp-config=<ไฟล์> (ต้องมี =)
  ห้าม symlink .claude.json (มีสถานะล็อกอิน); claude mcp list ไม่แสดง MCP นี้ ให้ตรวจด้วย /mcp
- Codex: skill อ่านจาก $CODEX_HOME/skills และเขียน skills/.system เอง ห้าม symlink ทั้งโฟลเดอร์
  MCP ส่งด้วย -c mcp_servers.<name>={...} (ใช้กับ codex mcp list ได้)
- Playwright ใน root container: --browser chromium --headless --isolated --no-sandbox และ --output-dir นอกโฟลเดอร์งาน
  ติดตั้ง Chromium ด้วย playwright ในแพ็กเกจ @playwright/mcp ให้เวอร์ชันตรงกัน (อาจต้อง install-deps)

## ทดสอบใน container (PASS/FAIL/SKIP ตามจริง)
- 10 คำสั่ง --version ครบ, config root คนละโฟลเดอร์, installer รอบ 2 ไม่ทำลายของเดิม
- cyoloN mcp list เห็น playwright, skill shared-check ทุกบัญชี readlink -f ไปไฟล์เดียวกันใน shared
- อะไรที่ต้องล็อกอิน = SKIP

## รายงาน (ห้ามมี secret)
- setup-report.md: โครงสร้างโฟลเดอร์, wrapper, แต่ละ CLI อ่าน skill/MCP จากไหน, ข้อจำกัด
- login-guide.md: ล็อกอินทีละบัญชีใน JupyterLab Terminal
- checklist.md: รายการ | หลักฐาน | PASS/FAIL/SKIP
เสร็จแล้วสรุปเป็นตารางสั้น ๆ แล้วหยุดรอ ห้ามล็อกอินแทนฉัน
```

ขั้นที่ 1.3 · 💻 Claude Code app

วางพรอมต์ → Enter → อนุญาตคำสั่ง docker



ภาพจำลองหน้าต่าง Claude Code app · ข้อความพรอมต์และคำตอบตรงกับการรันจริง

ขั้นที่ 1.3 · พรอมต์ (คัดลอกทั้งกรอบ)

```
ช่วยสร้าง Docker image ชื่อ multi-account:1.0 ที่เปิดมาแล้วใช้หลายบัญชี AI ได้ทันที
ทำในโฟลเดอร์นี้ ใช้คำสั่ง docker ได้ แต่ห้ามลบ หยุด หรือแก้ container / image / volume ที่มีอยู่
(ห้าม docker system prune และห้ามแตะ volume ai-accounts)

วัตถุดิบ
- ./00_Devtool_Container = Dockerfile มาตรฐานของ devtools คัดลอกเป็น ./MultiAccountDevtool แล้วแก้ในนั้นเท่านั้น
  ถ้ามี ./MultiAccountDevtool อยู่แล้ว ให้เทียบกับสเปกแล้วแก้เฉพาะที่ขาด
- ./prompt_multi_account.md = สเปกระบบหลายบัญชี ทุกข้อต้องเป็นจริงใน image
  ส่วน "ข้อเท็จจริงจากการทดลอง" ให้ตรวจกับ --help ของเวอร์ชันจริงก่อนใช้

สิ่งที่ต้องอยู่ใน image
- Claude Code, Codex CLI, Playwright MCP (pin เวอร์ชัน) + Chromium + ฟอนต์ไทย
- installer และคำสั่ง yolo1..5 / cyolo1..5 ใน /usr/local/bin ใช้ได้ทั้งใน JupyterLab Terminal และ docker exec
- ห้ามมีข้อมูลล็อกอินใน image: /root/ai-accounts เป็น volume และ installer รันทุกครั้งที่ container start โดยไม่ทับของเดิม
- คง Docker-in-Docker และ JupyterLab ไว้

build ด้วย: docker build -t multi-account:1.0 ./MultiAccountDevtool

ทดสอบด้วย container และ volume ทดสอบชื่อใหม่ แล้วลบทิ้งเมื่อเสร็จ (PASS/FAIL/SKIP ตามจริง)
- yolo1..5 / cyolo1..5 --version ครบ 10 คำสั่ง และ config root คนละโฟลเดอร์
- skill shared-check ทุกบัญชีชี้ไปไฟล์เดียวกันใน shared และ cyolo1 mcp list เห็น playwright
- Playwright MCP ผ่าน stdio เปิด https://example.com แล้ว screenshot ได้
- installer รอบ 2 ไม่ทับของเดิม และลบ container แล้วสร้างใหม่ ของใน volume ยังอยู่
- JupyterLab /lab ตอบ 200
ข้อที่ต้องล็อกอิน = SKIP ห้ามล็อกอินแทนผม

เขียนรายงานตามหัวข้อ "รายงาน" ของสเปกไว้ใน ./MultiAccountDevtool (ห้ามมี secret)
เสร็จแล้วสรุปเป็นตารางสั้น ๆ: ผลทดสอบ ขนาด image และเวลา build แล้วหยุด
```

ขั้นที่ 1.3

หลักการออกแบบพรอมต์

ระบุ input ชัดเจน

Claude ไม่ต้องคาดเดาฐานของ image หรือโครงสร้างระบบหลายบัญชี

แยก image ออกจาก volume

โปรแกรมอยู่ใน image ข้อมูลล็อกอินอยู่ใน volume การแชร์ image จึงไม่เปิดเผยบัญชี

ไม่แตะทรัพยากรเดิม

ทดสอบด้วย container และ volume ชั่วคราวแล้วลบทิ้ง

รายงานเป็น PASS/FAIL/SKIP

ตรวจผลจากหลักฐานได้ ไม่ต้องอาศัยคำบอกเล่าของ AI

ขั้นที่ 1.4 · ผลจริง 7 พ.ย. 2026

✻ **Claude** · ขั้นที่ 1 · build image ของเราเอง · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

สร้าง `multi-account:1.0` สำเร็จ เปิดมาแล้วพร้อมให้ล็อกอินหลายบัญชีได้เลย ผลทดสอบไม่มี FAIL ส่วนรายการ SKIP คือเรื่องที่ต้องล็อกอิน

ผลทดสอบ (image สุดท้าย)	ผล	หลักฐาน
`yolo1..5` / `cyolo1..5 --version`	PASS	ครบ 10 คำสั่ง: Claude Code 2.1.319, Codex 0.167.0
Config root แยกโฟลเดอร์	PASS	ไม่ซ้ำกันครบ 10 บัญชี
Skill `shared-check` ชี้ไฟล์เดียวกันใน shared	PASS	`readlink -f` ตรงทุกบัญชีของแต่ละ CLI
`cyolo1 mcp list` เห็น playwright	PASS	เห็นครบ `cyolo1..5`
Playwright MCP ผ่าน stdio เปิด example.com แล้ว screenshot	PASS	ได้ PNG ถูกต้อง และภาษาไทยแสดงผลถูก
Installer รอบ 2 ไม่ทับของเดิม	PASS	`สร้างใหม่ 0` และไฟล์ไม่เปลี่ยน
ลบ container แล้วสร้างใหม่ ของใน volume ยังอยู่	PASS	เทียบแล้วต่างกัน 0 บรรทัด
JupyterLab `/lab`	PASS	HTTP 200 และพิมพ์คำสั่งใน Terminal จริงได้
ไม่มีข้อมูลล็อกอินใน image	PASS	ไม่พบไฟล์ credential และค่าสุ่มของ skill
ล็อกอินจริง, `/mcp`, ให้โมเดลตอบ `shared-check`	SKIP	ต้องล็อกอิน

Image	ค่า
ขนาด	4.47GB
เวลา build (`--no-cache`)	302 วินาที (ประมาณ 5 นาที)

ของเดิม: container, volume และ image เดิมไม่เปลี่ยน ของทดสอบลบหมดแล้ว สิ่งที่เพิ่มในเครื่องคือ image นี้และ build cache

สิ่งที่ควรรู้

- **JupyterLab ไม่มีรหัสผ่าน:** Terminal ในนั้นเข้าถึง credential ทุกบัญชีได้ ตอนรันให้ผูก port กับ `127.0.0.1` หรือตั้ง `JUPYTER_PASSWORD`
- **รหัสผ่านโผล่ใน log:** `start.sh` ต้นฉบับพิมพ์ `JUPYTER_PASSWORD` ลง `docker logs` ถ้าตั้งไว้ ผมยังไม่ได้แก้เพราะอยู่นอกงานนี้
- **การแยกบัญชี:** แยกแค่ระดับโฟลเดอร์ config ไม่ใช่ OS isolation ทุกบัญชีรันเป็น root คนเดียวกัน จึงอ่าน credential ของกันได้
- **ต่างจากสเปกเดิม:** `--help` ของ Playwright MCP ไม่ระบุ `--browser chromium` แต่รันได้จริง และถ้าระบุชื่อไฟล์ screenshot เอง ไฟล์จะลงโฟลเดอร์งาน ไม่ใช่ `--output-dir`
- **การทดสอบเสริม:** ผมใช้ mock API และคีย์ปลอมใน volume ทดสอบ เพื่อดูว่า CLI เห็น skill และ MCP จริง ไม่ได้ล็อกอินบัญชีของคุณ
- **Volume `ai-accounts` :** ยังไม่ได้ทดสอบกับ image นี้ตามข้อห้าม ถ้าจะใช้ควรสำรองก่อน

ทุกอย่างอยู่ใน `MultiAccountDevtool/` เริ่มที่ `login-guide.md` สำหรับล็อกอินทีละบัญชี ส่วนผลละเอียดอยู่ใน `checklist.md` และโครงสร้างกับข้อจำกัดอยู่ใน `setup-report.md`

9 PASS **0 FAIL** **1 SKIP**

ขนาด image	**4.47 GB**
build แบบไม่ใช้ cache	**302 วินาที**
Claude Code	2.1.319
Codex CLI	0.167.0
Playwright MCP	0.0.84

SKIP คือข้อที่ต้องล็อกอิน · ตอนรันจริงใช้ชื่อ my-multi-account ภาพผลจริงปรับชื่อให้ตรงกับบทเรียน

ขั้นที่ 1.4 · ไฟล์ที่ Claude สร้าง

MultiAccountDevtool/

```
MultiAccountDevtool/
├── Dockerfile                ← Dockerfile ของ devtools + 4 ชั้นใหม่ (CLI, Playwright, ฟอนต์ไทย, ระบบหลายบัญชี)
├── start.sh                  ← dockerd + multi-account-install (ทุกครั้งที่ start) + jupyter lab
├── multi-account/            ← installer, wrapper ของ yoloN/cyoloN, ตัวแปลง MCP ให้ Codex
├── docker-compose.yml        ← ทางเลือก: build + run ด้วยคำสั่งเดียว
└── setup-report.md · login-guide.md · checklist.md   ← รายงาน 3 ไฟล์ตามสเปก
```

เก็บเป็นเฉลยไว้ที่ _solution/MultiAccountDevtool/

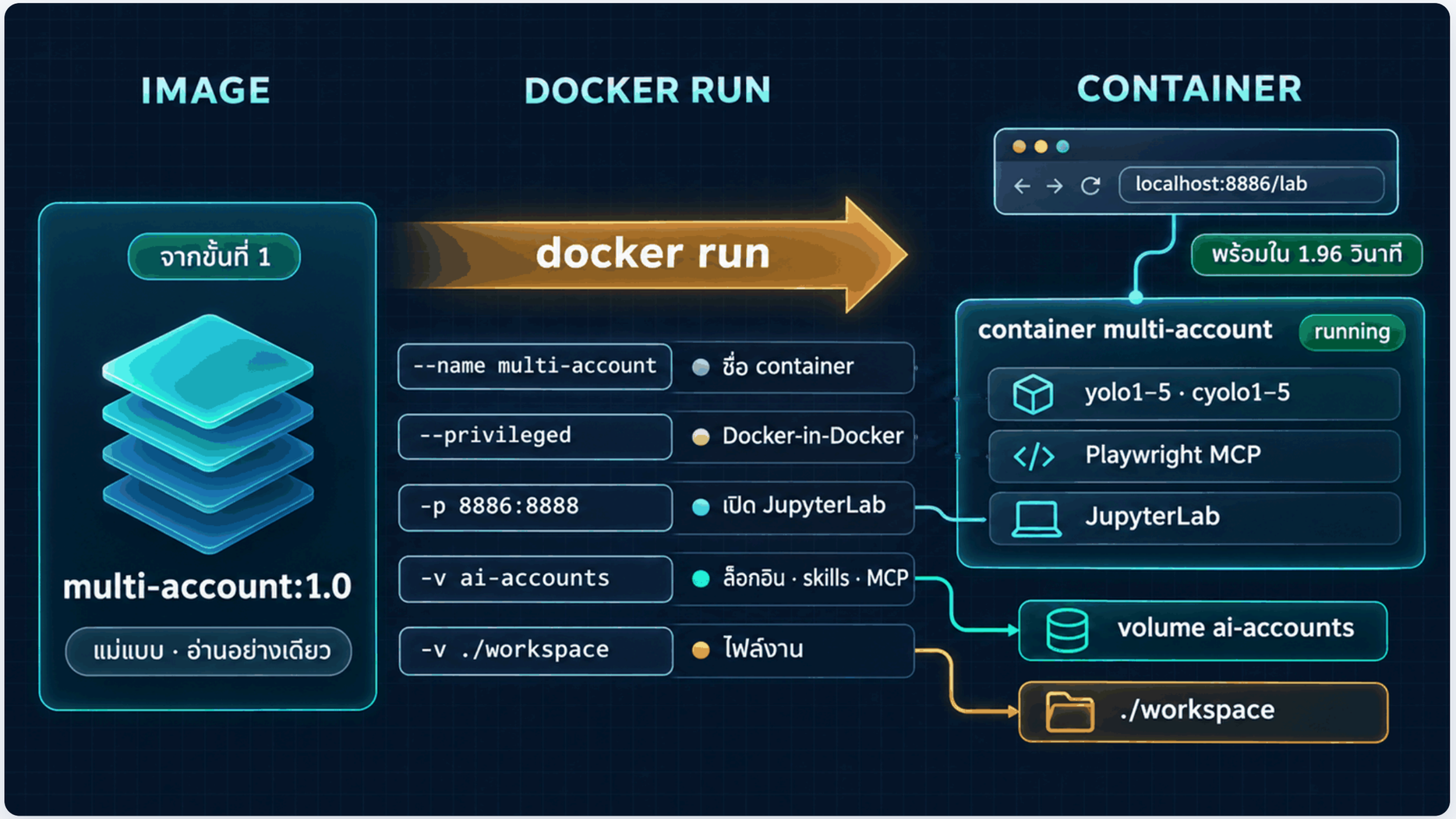
2

สร้าง container จาก image

image เป็นแม่แบบอ่านอย่างเดียว ต้อง **docker run** เป็น container ก่อนจึงใช้งานได้ (~1 นาที)

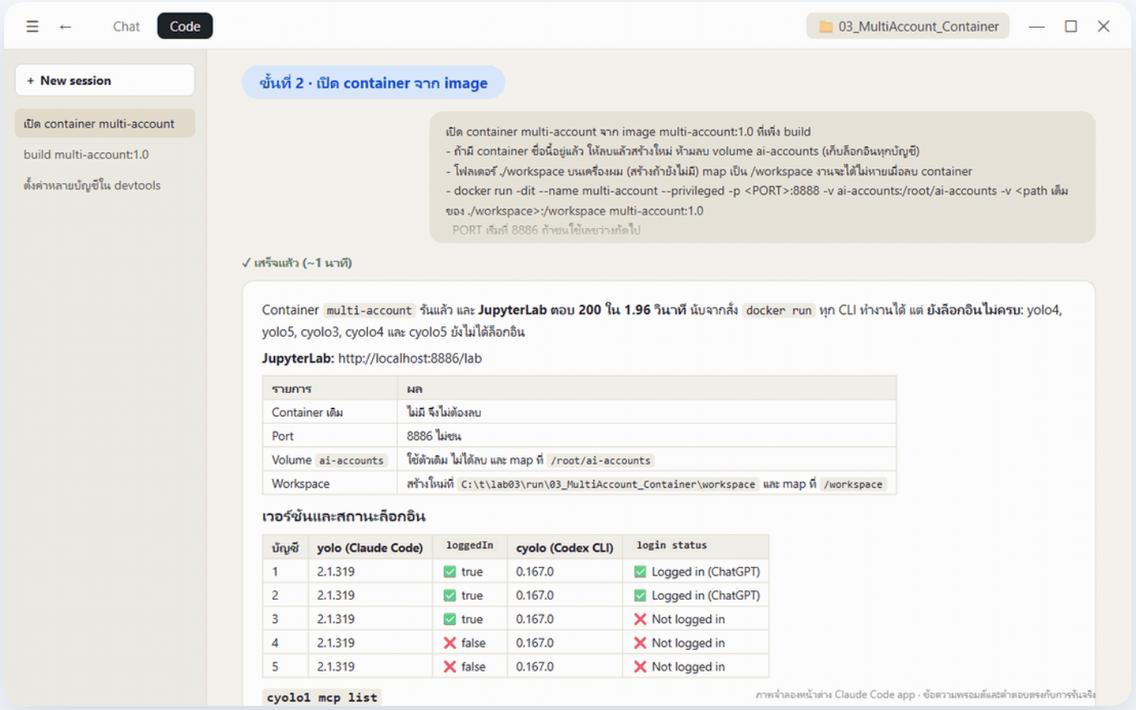
ขั้นที่ 2 · แนวคิด

แต่ละ option กำหนดว่า container เชื่อมกับเครื่องเราอย่างไร



ขั้นที่ 2 · 💻 Claude Code app

```
เปิด container multi-account จาก image multi-account:1.0 ที่เพิ่ง build
- ถ้ามี container ชื่อนี้อยู่แล้ว ให้ลบแล้วสร้างใหม่ ห้ามลบ volume ai-accounts (เก็บล็อกอินทุกบัญชี)
- โฟลเดอร์ ./workspace บนเครื่องผม (สร้างถ้ายังไม่มี) map เป็น /workspace งานจะได้ไม่หายเมื่อลบ
container
- docker run -dit --name multi-account --privileged -p <PORT>:8888 -v ai-
accounts:/root/ai-accounts -v <path เต็มของ ./workspace>:/workspace multi-account:1.0
  PORT เริ่มที่ 8886 ถ้าชนใช้เลขว่างถัดไป
- จับเวลาตั้งแต่สั่ง run จน http://localhost:<PORT>/lab ตอบ 200
- ตรวจผ่าน docker exec: yolo1..5 และ cyolo1..5 --version, cyolo1 mcp list,
  สถานะล็อกอิน (yoloN auth status ดูแค่ loggedIn / cyoloN login status) ห้ามแสดงอีเมลหรือ
token
สรุปเป็นตาราง พร้อม URL ของ JupyterLab และเวลาที่ใช้
```



ขั้นที่ 2 · ผลจริง

JupyterLab พร้อมใน 1.96 วินาที

✻ **Claude** · ขั้นที่ 2 · เปิด container จาก image · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

Container `multi-account` รันแล้ว และ **JupyterLab ตอบ 200 ใน 1.96 วินาที** นับจากสั่ง `docker run` ทุก CLI ทำงานได้ แต่ **ยังล็อกอินไม่ครบ**: yolo4, yolo5, cyolo3, cyolo4 และ cyolo5 ยังไม่ได้ล็อกอิน

JupyterLab: http://localhost:8886/lab

รายการ	ผล
Container เดิม	ไม่มี จึงไม่ต้องลบ
Port	8886 ไม่ชน
Volume `ai-accounts`	ใช้ตัวเดิม ไม่ได้ลบ และ map ที่ `/root/ai-accounts`
Workspace	สร้างใหม่ที่ `C:\t\lab03\run\03_MultiAccount_Container\workspace` และ map ที่ `/workspace`

เวอร์ชันและสถานะล็อกอิน

บัญชี	yolo (Claude Code)	`loggedIn`	cyolo (Codex CLI)	`login status`
1	2.1.319	✅ true	0.167.0	✅ Logged in (ChatGPT)
2	2.1.319	✅ true	0.167.0	✅ Logged in (ChatGPT)
3	2.1.319	✅ true	0.167.0	❌ Not logged in
4	2.1.319	❌ false	0.167.0	❌ Not logged in
5	2.1.319	❌ false	0.167.0	❌ Not logged in

`cyolo1 mcp list`

MCP server	Command	สถานะ	Auth
`playwright`	`/usr/bin/playwright-mcp` (chromium headless)	enabled	Unsupported
`rea`	`npx -y rea-agents@6.8.0 mcp`	enabled	Unsupported

คำสั่งนี้แค่บอกว่าเปิดใช้ server ไว้ ไม่ได้ทดสอบว่าเชื่อมต่อได้จริง

ข้อควรระวัง

- **ความปลอดภัย:** `/lab` ตอบ 200 โดยไม่ต้องใส่ token และ port เปิดที่ `0.0.0.0` คนอื่นในเครือข่ายเดียวกันจึงอาจเข้า JupyterLab ได้ด้วย ถ้า Terminal ของ JupyterLab เปิดใช้อยู่ เขาจะสั่งงานใน container แบบ `--privileged` ที่มีบัญชีล็อกอินอยู่ได้ ถ้าไม่ต้องการให้เข้าจากเครื่องอื่น ให้เปลี่ยนเป็น `-p 127.0.0.1:8886:8888`
- **Volume ที่จะค้าง:** image สร้าง anonymous volume สำหรับ `/var/lib/docker` เพิ่มอีกตัว ถ้าลบด้วย `docker rm -f` เฉยๆ volume นี้จะค้างอยู่ ส่วน `docker rm -v` จะลบเฉพาะตัวนี้ ไม่แตะ `ai-accounts` ซึ่งเป็น named volume

> คำสั่งครบ 10 ตัว และ `cyolo1 mcp list` แสดง `playwright` ทันที เพราะโปรแกรมทั้งหมดอยู่ใน image แล้ว

เครื่องตัวอย่างใช้ volume ที่เคยล็อกอินไว้แล้ว 5 บัญชี ของผู้เรียนจะเป็น ❌ ทั้งหมดจนกว่าจะทำขั้นที่ 3

3

ล็อกอินและให้ Claude ตรวจสถานะ

ล็อกอินเองใน JupyterLab ครั้งเดียว ข้อมูลเก็บใน **volume ai-accounts** (~15 นาที)

ขั้นที่ 3.1 · 🖥️ JupyterLab · 1 บัญชีต่อ 1 แท็บ Terminal

อย่างน้อย yolo1, yolo2, cyolo1, cyolo2

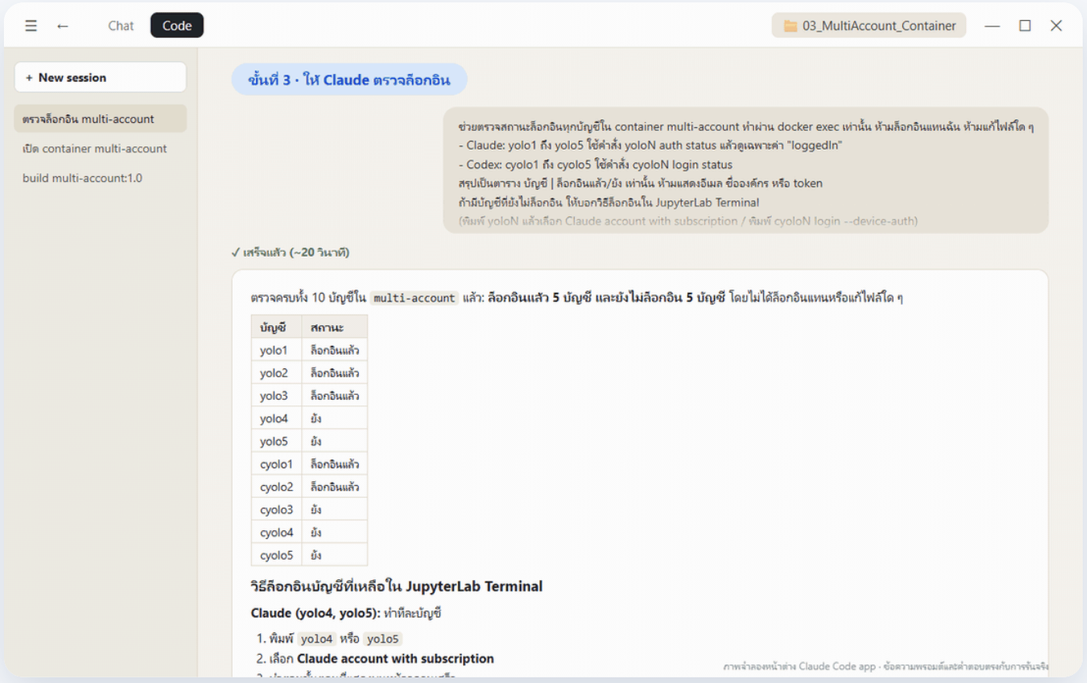
พิมพ์	เลือก
`yolo1` ... `yolo5`	**Claude account with subscription** → ยืนยันในเบราว์เซอร์ → **Yes, I trust this folder** → `/exit`
`cyolo1` ... `cyolo5`	**Sign in with ChatGPT** หรือ `cyolo1 login --device-auth`

ต่างจากแบบที่ 2: ล็อกอินเก็บใน volume จึงล็อกอินครั้งเดียว และยังใช้ได้แม้ลบ container หรือ build image ใหม่

ผู้เรียนต้องล็อกอินด้วยตนเองเสมอ ห้ามส่งรหัสผ่าน code หรือ URL ที่มี token ให้ AI หรือผู้อื่น

ขั้นที่ 3.2 · 💻 Claude Code app

```
ช่วยตรวจสถานะล็อกอินทุกบัญชีใน container multi-account ทำผ่าน docker exec เท่านั้น ห้ามล็อกอินแทนฉัน ห้ามแก้ไฟล์ใด ๆ
- Claude: yolo1 ถึง yolo5 ใช้คำสั่ง yoloN auth status แล้วดูเฉพาะค่า "loggedIn"
- Codex: cyolo1 ถึง cyolo5 ใช้คำสั่ง cyoloN login status
สรุปเป็นตาราง บัญชี | ล็อกอินแล้ว/ยัง เท่านั้น ห้ามแสดงอีเมล ชื่อองค์กร หรือ token
ถ้ามีบัญชีที่ยังไม่ล็อกอิน ให้บอกวิธีล็อกอินใน JupyterLab Terminal
(พิมพ์ yoloN แล้วเลือก Claude account with subscription / พิมพ์ cyoloN login --device-auth)
```



ภาพจำลองหน้าต่าง Claude Code app · ข้อความพร้อมคัดลอกด้านล่างเป็นคำตอบจริงจากการรันจริง

✻ Claude · ขั้นที่ 3 · ให้ Claude ตรวจล็อกอิน · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ตรวจครบทั้ง 10 บัญชีใน `multi-account` แล้ว: **ล็อกอินแล้ว 5 บัญชี และยังไม่ล็อกอิน 5 บัญชี** โดยไม่ได้ล็อกอินแทนหรือแก้ไฟล์ใด ๆ

บัญชี	สถานะ
yolo1	ล็อกอินแล้ว
yolo2	ล็อกอินแล้ว
yolo3	ล็อกอินแล้ว
yolo4	ยัง
yolo5	ยัง
cyolo1	ล็อกอินแล้ว
cyolo2	ล็อกอินแล้ว
cyolo3	ยัง
cyolo4	ยัง
cyolo5	ยัง

วิธีล็อกอินบัญชีที่เหลือใน JupyterLab Terminal

Claude (yolo4, yolo5): ทำทีละบัญชี

1. พิมพ์ `yolo4` หรือ `yolo5`
2. เลือก **Claude account with subscription**
3. ทำตามขั้นตอนที่แสดงบนหน้าจอจนเสร็จ

Codex (cyolo3, cyolo4, cyolo5): พิมพ์คำสั่งของแต่ละบัญชี แล้วทำตามขั้นตอนที่แสดงบนหน้าจอ `bash cyolo3 login --device-auth cyolo4 login --device-auth cyolo5 login --device-auth`

ผลจริง ~20 วินาที · ขั้นที่ 4 ต้องมีอย่างน้อย 1 บัญชีต่อ CLI

4

ทดสอบ skill และ MCP ข้ามบัญชีแบบขนาน

ทุกบัญชีเรียก **shared-check** และใช้ **Playwright MCP** ค้นหาเที่ยวบิน BKK → Tokyo พร้อมกัน (~5 นาที)

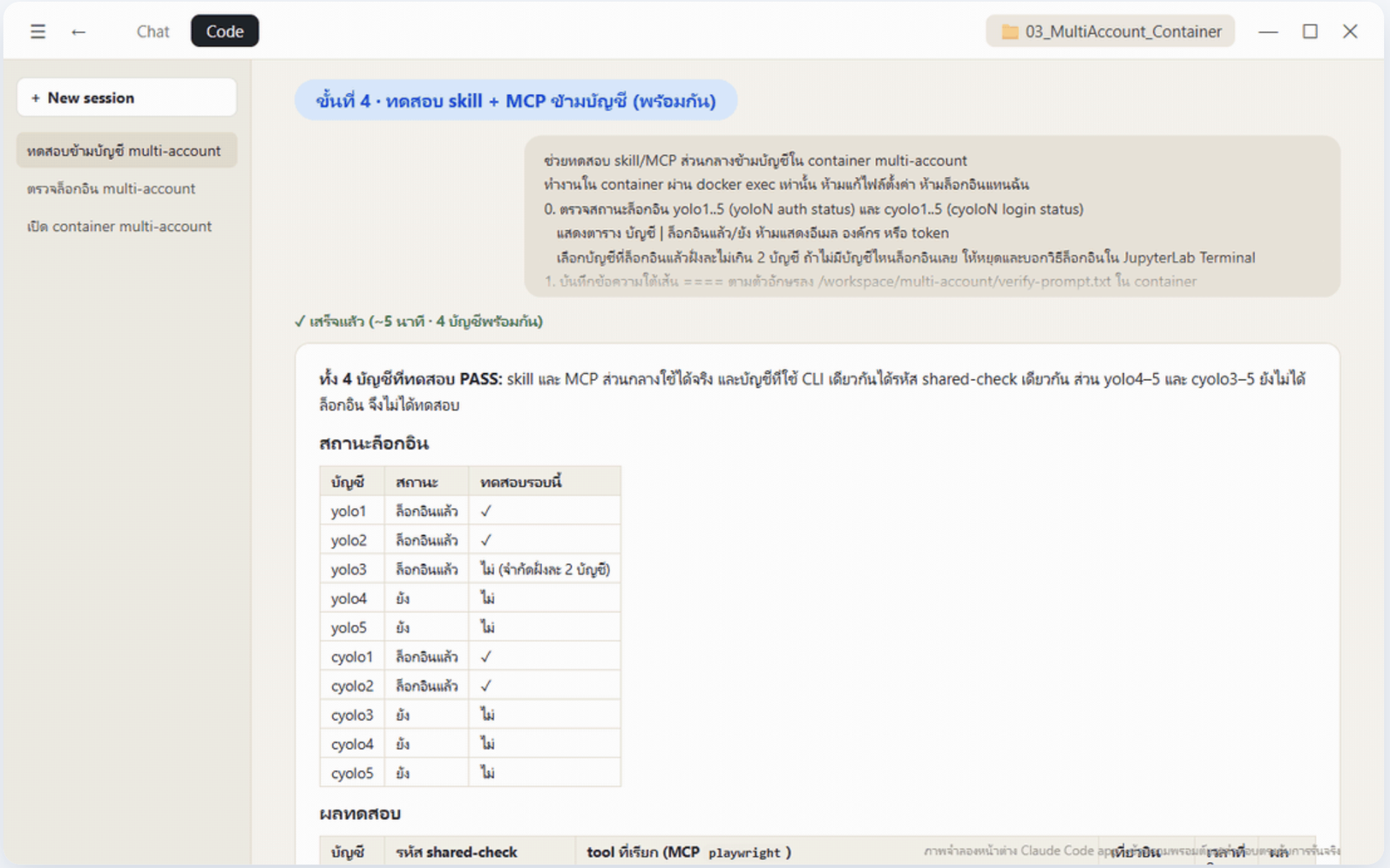
ขั้นที่ 4 · พรอมต์ (เหนือ ==== งานของ Claude · ใต้ ==== งานของแต่ละบัญชี)

```
ช่วยทดสอบ skill/MCP ส่วนกลางข้ามบัญชีใน container multi-account
ทำงานใน container ผ่าน docker exec เท่านั้น ห้ามแก้ไฟล์ตั้งค่า ห้ามล็อกอินแทนฉัน
0. ตรวจสถานะล็อกอิน yolo1..5 (yoloN auth status) และ cyolo1..5 (cyoloN login status)
   แสดงตาราง บัญชี | ล็อกอินแล้ว/ยัง ห้ามแสดงอีเมล องค์กร หรือ token
   เลือกบัญชีที่ล็อกอินแล้วฝั่งละไม่เกิน 2 บัญชี ถ้าไม่มีบัญชีไหนล็อกอินเลย ให้หยุดและบอกวิธีล็อกอินใน JupyterLab Terminal
1. บันทึกข้อความใต้เส้น ==== ตามตัวอักษรลง /workspace/multi-account/verify-prompt.txt ใน container
2. รันทุกบัญชีที่เลือกพร้อมกันจากโฟลเดอร์ /workspace/multi-account แล้วรอจนครบทุกบัญชี (timeout บัญชีละ 15 นาที):
   yoloN -p "$(cat verify-prompt.txt)" / cyoloN exec --skip-git-repo-check "$(cat verify-prompt.txt)"
3. คัดลอกภาพที่แต่ละบัญชีรายงานไปเป็น /workspace/multi-account/evidence/<บัญชี>/1-open.png, 2-search.png, 3-results.png
   พร้อม log คำตอบเป็น run.log ในโฟลเดอร์เดียวกัน
4. สรุปตาราง บัญชี | รหัส shared-check | tool ที่เรียก | เที่ยวบินอันดับ 1 | เวลาที่ใช้ | PASS/FAIL
   บัญชีต่างกันแต่ CLI เดียวกันต้องได้รหัสเดียวกัน ถ้าบัญชีใดลืมเหลวให้รายงานตามจริง ห้ามแก้แทนหรือเดาผล
====
ตรวจว่า skill และ MCP ส่วนกลางใช้ได้จริงในบัญชีนี้ ทำตามลำดับ ห้ามแก้ไฟล์ตั้งค่าใด ๆ

1. แจ้ง config root ที่ใช้อยู่ (ค่า CLAUDE_CONFIG_DIR หรือ CODEX_HOME) ห้ามแสดง token อีเมล หรือข้อมูลบัญชี
2. ใช้ skill shared-check ตามกลไก skill ของ CLI นี้ แล้วรายงานรหัสที่ได้ พร้อม path ของ skill ตามที่ CLI แสดง
   ห้ามค้นหาไฟล์เอง (find, grep, ls) และห้ามเปิดไฟล์จาก path ที่ไม่ได้มาจากรายการ skill ของ CLI
3. ใช้เครื่องมือของ MCP ชื่อ playwright เท่านั้น ห้ามใช้ curl, python, web search หรือ browser อื่น
   เปิด https://www.google.com/travel/flights?hl=en
   ค้นหาเที่ยวบินเที่ยวเดียว Bangkok (BKK) ไป Tokyo ญี่ปุ่น ชั้นประหยัด ผู้ใหญ่ 1 คน วันเดินทางคือวันที่ 15 ของเดือนถัดไป
   ค้นหาอย่างเดียว ห้ามกดจอง ห้ามกรอกข้อมูลส่วนตัว ห้ามชำระเงิน ห้ามล็อกอินเว็บไซต์ใด
   ถ้าหน้าเว็บไม่แสดงผล ให้รายงานตามจริงว่าติดตรงไหน ห้ามเดาราคา
4. บันทึกภาพหน้าจอด้วย tool screenshot ของ playwright เฉพาะส่วนที่มองเห็น 3 ภาพ
   ตั้งชื่อ <บัญชี>-1-open.png (หน้าเว็บเปิดได้), <บัญชี>-2-search.png (กรอกเงื่อนไขครบก่อนกดค้นหา),
   <บัญชี>-3-results.png (รายการผลเที่ยวบิน) แล้วรายงาน path เต็มของแต่ละภาพ
5. สรุปเป็นตาราง รายการ | PASS/FAIL | หลักฐาน โดยมีแถว: config root, รหัสจาก skill shared-check,
   ชื่อ MCP server และชื่อ tool ที่เรียกจริง, ภาพหน้าจอ 3 ภาพ (path),
   เที่ยวบิน 3 อันดับแรก: สายการบิน เวลาออก-ถึง ระยะเวลา จำนวนแวะ ราคา (ตามที่เห็นบนหน้าเว็บ)
```

ขั้นที่ 4 · 💻 Claude Code app

4 บัญชีทำงานพร้อมกัน



ค้นหาอย่างเดียว ไม่จองและไม่ชำระเงิน

ขั้นที่ 4 · ผลจริง 7 พ.ย. 2026

5 นาที 20 วินาที สำหรับ 4 บัญชีพร้อมกัน

✻ Claude · ขั้นที่ 4 · ทดสอบ skill + MCP ข้ามบัญชี (พร้อมกัน) · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ทั้ง 4 บัญชีที่ทดสอบ PASS: skill และ MCP ส่วนกลางใช้ได้จริง และบัญชีที่ใช้ CLI เดียวกันได้รหัส shared-check เดียวกัน ส่วน yolo4–5 และ cyolo3–5 ยังไม่ได้ล็อกอิน จึงไม่ได้ทดสอบ

สถานะล็อกอิน

บัญชี	สถานะ	ทดสอบรอบนี้
yolo1	ล็อกอินแล้ว	✓
yolo2	ล็อกอินแล้ว	✓
yolo3	ล็อกอินแล้ว	ไม่ (จำกัดฝั่งละ 2 บัญชี)
yolo4	ยัง	ไม่
yolo5	ยัง	ไม่
cyolo1	ล็อกอินแล้ว	✓
cyolo2	ล็อกอินแล้ว	✓
cyolo3	ยัง	ไม่
cyolo4	ยัง	ไม่
cyolo5	ยัง	ไม่

ผลทดสอบ

บัญชี	รหัส shared-check	tool ที่เรียก (MCP `playwright`)	เที่ยวบินอันดับ 1	เวลาที่ใช้	ผล
yolo1	`SHARED_SKILL_OK-claude-84dd0687`	`browser_navigate`, `browser_snapshot`, `browser_click`, `browser_type`, `browser_wait_for`, `browser_take_screenshot`	Vietjet, THB 5,956	175 วิ	**PASS**
yolo2	`SHARED_SKILL_OK-claude-84dd0687`	ชุดเดียวกับ yolo1 และเพิ่ม `browser_hover`, `browser_close`	Vietjet, THB 5,956	160 วิ	**PASS**
cyolo1	`SHARED_SKILL_OK-codex-4e38203a`	`browser_navigate`, `browser_snapshot`, `browser_find`, `browser_click`, `browser_type`, `browser_take_screenshot`, `browser_run_code_unsafe`	Vietjet, THB 5,956	108 วิ	**PASS**
cyolo2	`SHARED_SKILL_OK-codex-4e38203a`	`browser_navigate`, `browser_snapshot`, `browser_find`, `browser_click`, `browser_type`, `browser_take_screenshot`	Vietjet, THB 5,956	98 วิ	**PASS**

- **เที่ยวบินอันดับ 1:** ออก 14:00 ถึง 01:00 (+1 วัน), 9 ชม., แวะ 1 ครั้งที่ SGN, BKK–HND
- **เงื่อนไขค้นหา:** วันที่ 15 ธ.ค. 2026 ทุกบัญชี ส่วนราคาเป็นราคาบนหน้าเว็บ ณ เวลาที่ค้น

(ตัดส่วน "สิ่งที่ตรวจเองเพิ่ม" และ "ข้อสังเกต" ท้ายคำตอบออกเพื่อให้อ่านง่าย)

บัญชี	รหัส shared-check	MCP	เวลา
yolo1	`…claude-84dd0687`	PASS	175 วินาที
yolo2	เหมือน yolo1	PASS	160 วินาที
cyolo1	`…codex-4e38203a`	PASS	108 วินาที
cyolo2	เหมือน cyolo1	PASS	98 วินาที

เที่ยวบินอันดับ 1 ทุกบัญชี: Vietjet · 9 ชม. · แวะ 1 · THB 5,956 · แบบที่ 2 รันทีละบัญชีใช้ ~10 นาที

ขั้นที่ 4

การตีความผล

skill กลางใช้ได้

บัญชีต่างกันแต่ CLI เดียวกันได้รหัสเดียวกัน แสดงว่าอ่านไฟล์เดียวกันใน shared

MCP กลางใช้ได้

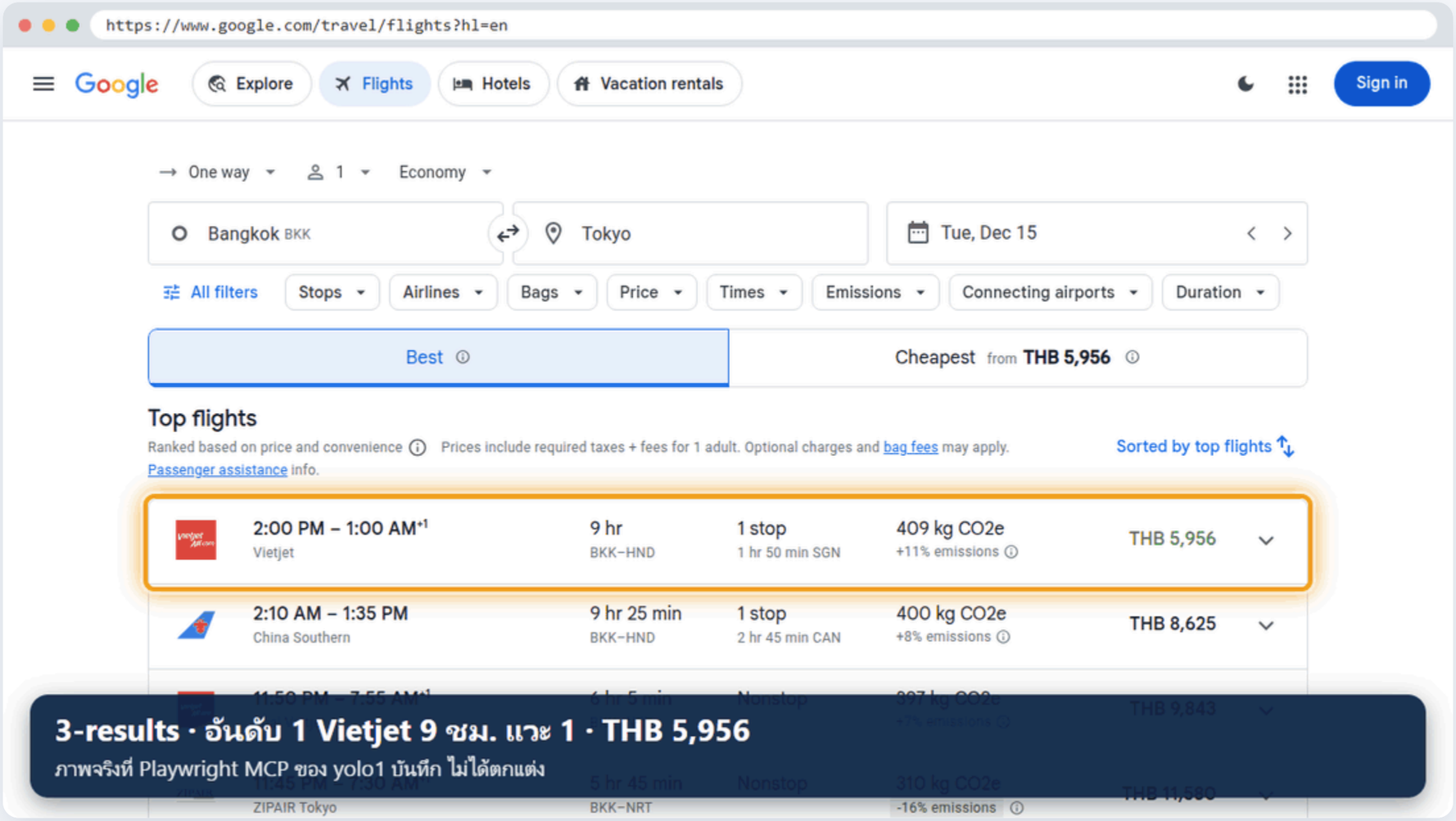
ทุกบัญชีเรียก tool ของ playwright ได้เอง เช่น browser_navigate, browser_take_screenshot

ทำงานพร้อมกันได้

4 บัญชีใช้ Chromium แยกกันใน container เดียวโดยไม่ชนกัน

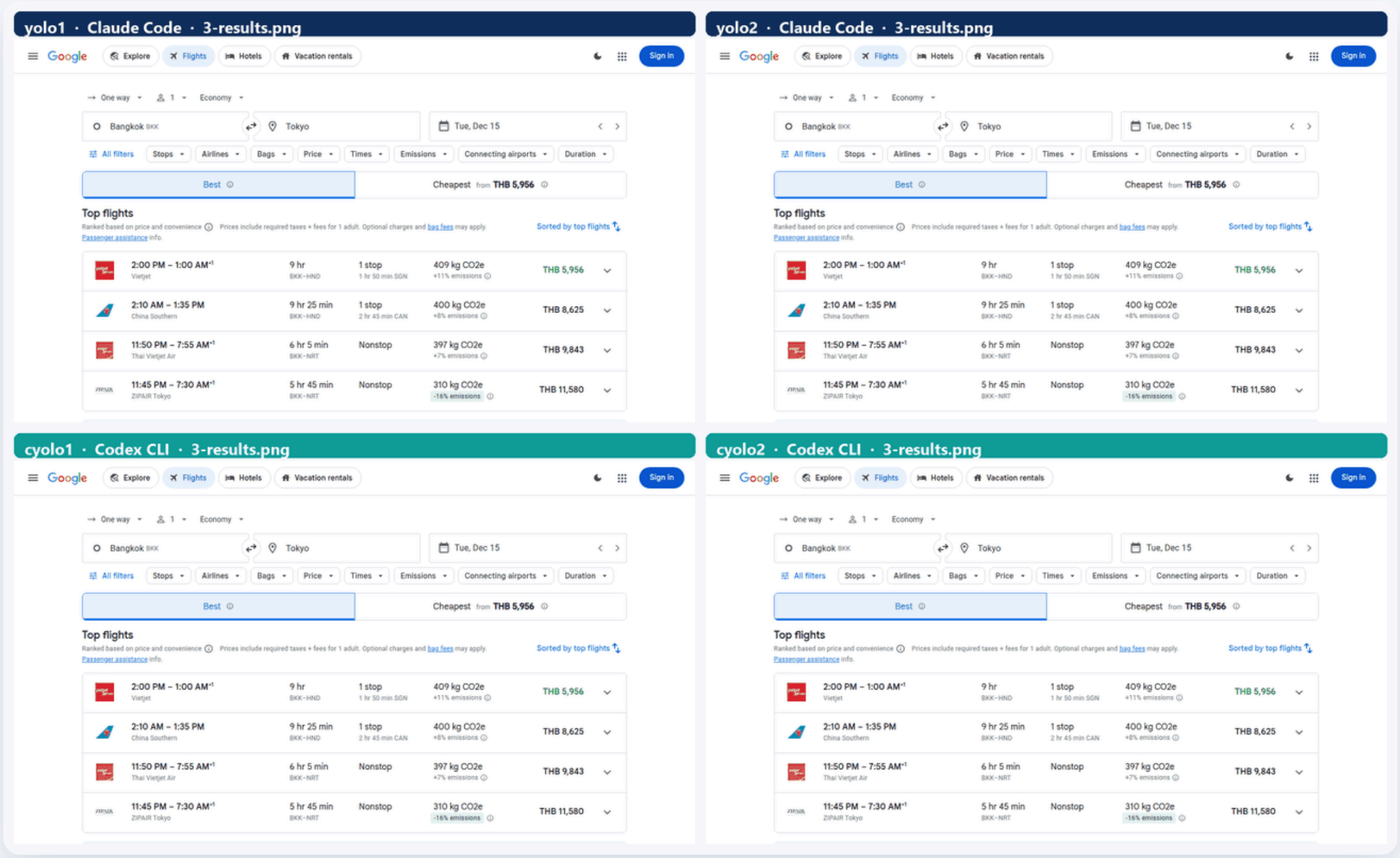
หลักฐานอยู่บนเครื่องที่ ./workspace/multi-account/evidence/<บัญชี>/ เพราะ map ./workspace ไว้ในขั้นที่ 2

ขั้นที่ 4 · ภาพจริงจาก Playwright MCP ของ yolo1



ขั้นที่ 4 · ภาพจริงที่แต่ละบัญชีบันทึกเอง

4 บัญชีได้ผลชุดเดียวกัน



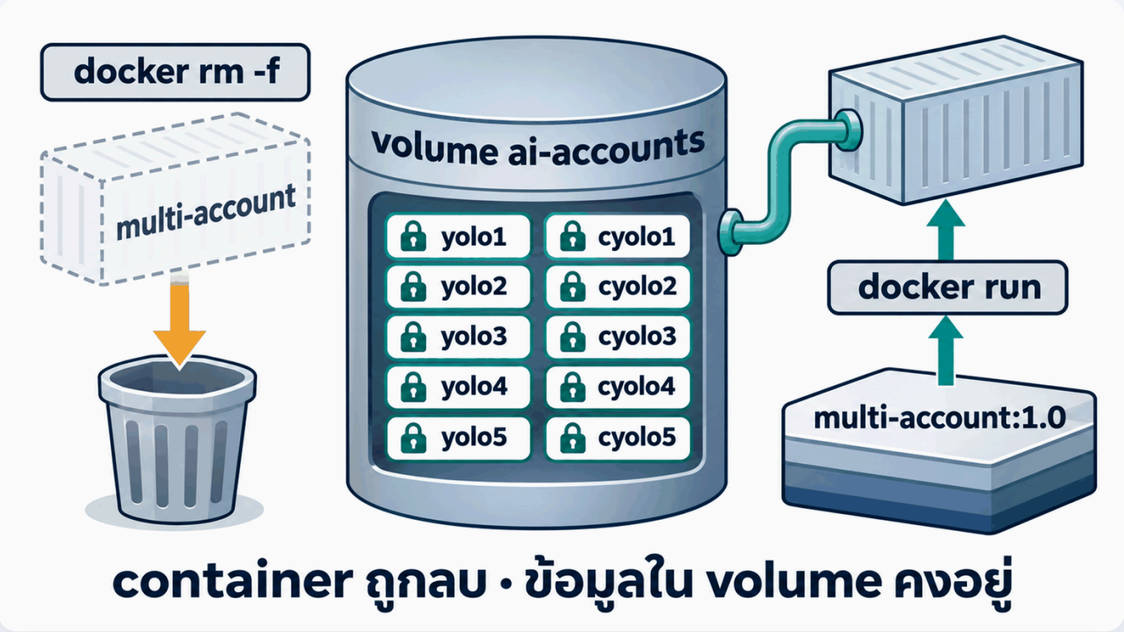
ราคาของผู้เรียนจะต่างตามวันที่ค้นหา

5

ทดสอบความคงทน: ลบ container แล้วสร้างใหม่

ในแบบที่ 2 ต้องติดตั้งและล็อกอินใหม่ทั้งหมด แบบที่ 3 คืนสภาพได้อย่างไร (~3 นาที)

ขั้นที่ 5 · 💻 Claude Code app




```
พิสูจน์ว่าทุกอย่างถูกอบไว้ใน image แล้ว ด้วยการลบ container multi-account แล้วสร้างใหม่
- ก่อนลบ: บันทึกสถานะล็อกอินทุกบัญชี (ห้ามแสดงอีเมลหรือ token)
- ลบด้วย docker rm -f แล้วรัน docker run คำสั่งเดิม (port เดิม volume ai-accounts เดิม
./workspace เดิม)
  ห้ามลบ volume ห้ามติดตั้งอะไรเพิ่ม จับเวลาตั้งแต่ลบจน JupyterLab ตอบ 200
- หลังสร้างใหม่ ตรวจผ่าน docker exec: yolo1..5 และ cyolo1..5 --version, cyolo1 mcp list
เห็น playwright,
  skill shared-check ยังอยู่, สถานะล็อกอินเทียบกับก่อนลบ, ไฟล์ใน /workspace/multi-
account/evidence ยังอยู่
- ให้บัญชีแรกที่ล็อกอินแล้วของแต่ละฝั่งตอบสั้น ๆ ว่า "พร้อม" (yoloN -p / cyoloN exec --skip-git-
repo-check) เพื่อยืนยันว่าใช้งานได้จริงทันที
สรุปเป็นตาราง ก่อนลบ | หลังสร้างใหม่ และเวลาที่ใช้
```

ขั้นที่ 5 · ผลจริง 7 พ.ย. 2026

3.27 วินาที ตั้งแต่ลบจนพร้อมใช้



✻ Claude · ขั้นที่ 5 · ลบ container แล้วสร้างใหม่ · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

สร้าง container ใหม่สำเร็จและใช้งานได้ทันที แต่พิสูจน์ไม่ได้ว่า "ทุกอย่างอบไว้ใน image" เครื่องมือหลักอยู่ใน image จริง แต่ MCP `rea` ต้องดาวน์โหลดใหม่ทุกครั้งที่สร้าง container ส่วนสถานะล็อกอินและ skill อยู่ใน volume และ evidence อยู่ในโฟลเดอร์ `./workspace` ที่ mount เข้าไป

เวลาตั้งแต่ลบจน JupyterLab ตอบ 200: 3.27 วินาที (ลบ 0.84 + สร้าง 0.71 + รอ 200 อีก 1.73)

รายการ	ก่อนลบ	หลังสร้างใหม่	เก็บอยู่ที่
Container ID	`46af07d5edfb`	`2930f7cb74cb` (ตัวใหม่)	–
Image ID	`sha256:d7052b3169da…`	เหมือนเดิม	–
ค่าการรัน	`--privileged`, 8886→8888, `ai-accounts`, `./workspace`	เหมือนเดิม	–
`yolo1..5 --version`	2.1.319 ครบ 5 ตัว	เหมือนเดิม	image
`cyolo1..5 --version`	0.167.0 ครบ 5 ตัว	เหมือนเดิม	image
`cyolo1 mcp list`	เห็น `playwright`	เหมือนเดิม	โปรแกรมอยู่ใน image, ค่าตั้งอยู่ใน volume
Skill `shared-check`	มีครบ ลิงก์ถูกทั้ง 10 บัญชี	hash และลิงก์เหมือนเดิม	volume
`yolo1`, `yolo2`, `yolo3`	ล็อกอินแล้ว (claude.ai)	เหมือนเดิม	volume
`yolo4`, `yolo5`	ยังไม่ล็อกอิน	เหมือนเดิม	–
`cyolo1`, `cyolo2`	ล็อกอินแล้ว (ChatGPT)	เหมือนเดิม	volume
`cyolo3`, `cyolo4`, `cyolo5`	ยังไม่ล็อกอิน	เหมือนเดิม	–
ไฟล์ใน `evidence`	16 ไฟล์	16 ไฟล์ hash รวมเหมือนเดิม	`./workspace`
ไฟล์ใน volume บัญชี	10,788 ไฟล์	10,788 ไฟล์	volume
`yolo1 -p`	ไม่ได้ทดสอบ	ตอบ "พร้อม" (13.5 วินาที)	–
`cyolo1 exec --skip-git-repo-check`	ไม่ได้ทดสอบ	ตอบ "พร้อม" (5.6 วินาที)	–

ผลตรวจอัตโนมัติก่อนและหลังตรงกันทุกบรรทัด ส่วนการทดสอบตอบ "พร้อม" ห่าเฉพาะหลังสร้างใหม่

(ตัดส่วน "สิ่งที่ต้องรู้" และ "คำถามสำหรับคุณ" ท้ายคำตอบออก ซึ่งเป็นเรื่อง MCP `rea` ที่เครื่องตัวอย่างเคยเพิ่มไว้ใน volume จากแลปอื่น)

ลบ 0.84 + สร้าง 0.71 + รอ 1.73 วินาที · ทุกรายการเหมือนก่อนลบ · yolo1 / cyolo1 ตอบ "พร้อม" ได้ทันทีโดยไม่ต้องล็อกอินใหม่

ขั้นที่ 5 · แบบที่ 2 เทียบกับแบบที่ 3

~20 นาที เทียบกับ 3.27 วินาที



ตัวเลขของแบบที่ 3 วัดจริงในขั้นนี้ · แบบที่ 2 มาจากบท 02_Setup_Multi_Account และเวลาโดยประมาณของการล็อกอิน


ภาพจำลอง
multi-account:1.0
yolo1-5 OK
cyolo1-5 OK
playwright OK
ready in 3.27 s
multi-account:1.0
yolo1-5 OK
cyolo1-5 OK
playwright OK
ready in 3.27 s
multi-account:1.0
yolo1-5 OK
cyolo1-5 OK
playwright OK
ready in 3.27 s
image เดียวกัน · สภาพแวดล้อมเหมือนกันทุกเครื่อง
โปรแกรมอยู่ใน image · ล็อกอินอยู่ใน volume · ไฟล์งานอยู่ใน ./workspace

สรุป

แบบที่ 2 เทียบกับแบบที่ 3

	แบบที่ 2 · ติดตั้งใน container ที่กำลังทำงาน	แบบที่ 3 · build จาก Dockerfile
ระบบหลายบัญชีอยู่ที่	ใน container `devtools`	ใน image `multi-account:1.0`
ข้อมูลล็อกอินอยู่ที่	ใน container (หายเมื่อลบ)	ใน volume `ai-accounts` (คงอยู่)
เมื่อลบ container	ติดตั้งใหม่ + ล็อกอินใหม่	`docker run` ใหม่ พร้อมใช้ใน 3.27 วินาที
เครื่องใหม่ / แชร์ให้ทีม	ทำซ้ำทุกเครื่อง ผลอาจต่างกัน	build จาก Dockerfile เดียวกัน หรือ push image ให้ทีม pull

ข้อควรรู้

ข้อควรรู้

ล็อกอินอยู่ใน volume

`docker rm -f` ปลอดภัย แต่ `docker volume rm ai-accounts` หรือ `prune --volumes` ลบล็อกอินทุกบัญชี

JupyterLab ไม่มีรหัสผ่าน

ใช้ `-p 127.0.0.1:8886:8888` หรือตั้ง `JUPYTER_PASSWORD`

image ต้องไม่มีล็อกอิน

ตรวจทุกครั้งก่อน push ขึ้น Docker Hub

แยกบัญชีระดับโฟลเดอร์

ทุกบัญชีทำงานเป็น root คนเดียวกัน ไม่ใช่การแยกสิทธิ์ระดับ OS

volume ของ Docker-in-Docker

ใช้ `docker rm -f -v multi-account` หากต้องการลบ volume ของ /var/lib/docker ด้วย

image สำเร็จรูป

`tuchsanai/multi-account-devtool:2569_1` ผ่านสกิล `/maccount`

คำสั่งที่ใช้บ่อย

บนเครื่อง

```
docker logs -f multi-account          # ดู log ตอน start
docker exec -it multi-account bash    # เข้า shell ใน container
docker stop multi-account             # หยุด
docker start multi-account            # เริ่มใหม่
docker rm -f multi-account            # ลบ container (ล็อกอินยัง
อยู่ใน volume ai-accounts)
docker build -t multi-account:1.0
_solution/MultiAccountDevtool   # build จากเฉลย
```

ใน container (JupyterLab Terminal)

```
yolo1 auth status         # สถานะล็อกอิน Claude
cyolo1 login status       # สถานะล็อกอิน Codex
cyolo1 mcp list           # ตรวจ MCP ของ Codex
multi-account-install     # รัน installer ซ้ำ (ไม่ทับของเดิม)
```

เช็กลิสต์

ทำครบหรือยัง?

- [ ] มี image `multi-account:1.0` (ขั้นที่ 1)
- [ ] container `multi-account` เปิด JupyterLab ได้ (ขั้นที่ 2)
- [ ] ล็อกอินอย่างน้อย yolo 1 + cyolo 1 บัญชี และ Claude ตรวจผ่าน (ขั้นที่ 3)
- [ ] ทดสอบพร้อมกันได้ PASS และมีภาพใน `./workspace/multi-account/evidence/` (ขั้นที่ 4)
- [ ] ลบแล้วสร้างใหม่ ล็อกอินยังอยู่ (ขั้นที่ 5)

สรุป

build ครั้งเดียว ใช้ได้ทุกเครื่อง

โปรแกรมอยู่ใน **image** · ล็อกอินอยู่ใน **volume** · ไฟล์งานอยู่ใน **./workspace**
container จึงลบและสร้างใหม่ได้โดยไม่สูญเสียข้อมูล

รายละเอียดทุกขั้นตอนและพรอมต์เต็มอยู่ใน README.md

